

Двухбашенные нейросети

ШАД, Рекомендательные системы
2025/26, лекция 4

Кирилл Хрыльченко

Старший преподаватель,
ФКН ВШЭ

План занятия

На лекции:

1. Введение в двухбашенные модели
2. Софтмакс модель
3. Функции близости и температура
4. Сэмплирование негативов

На семинаре разбор статей про logQ correction и техники сэмплирования негативов.

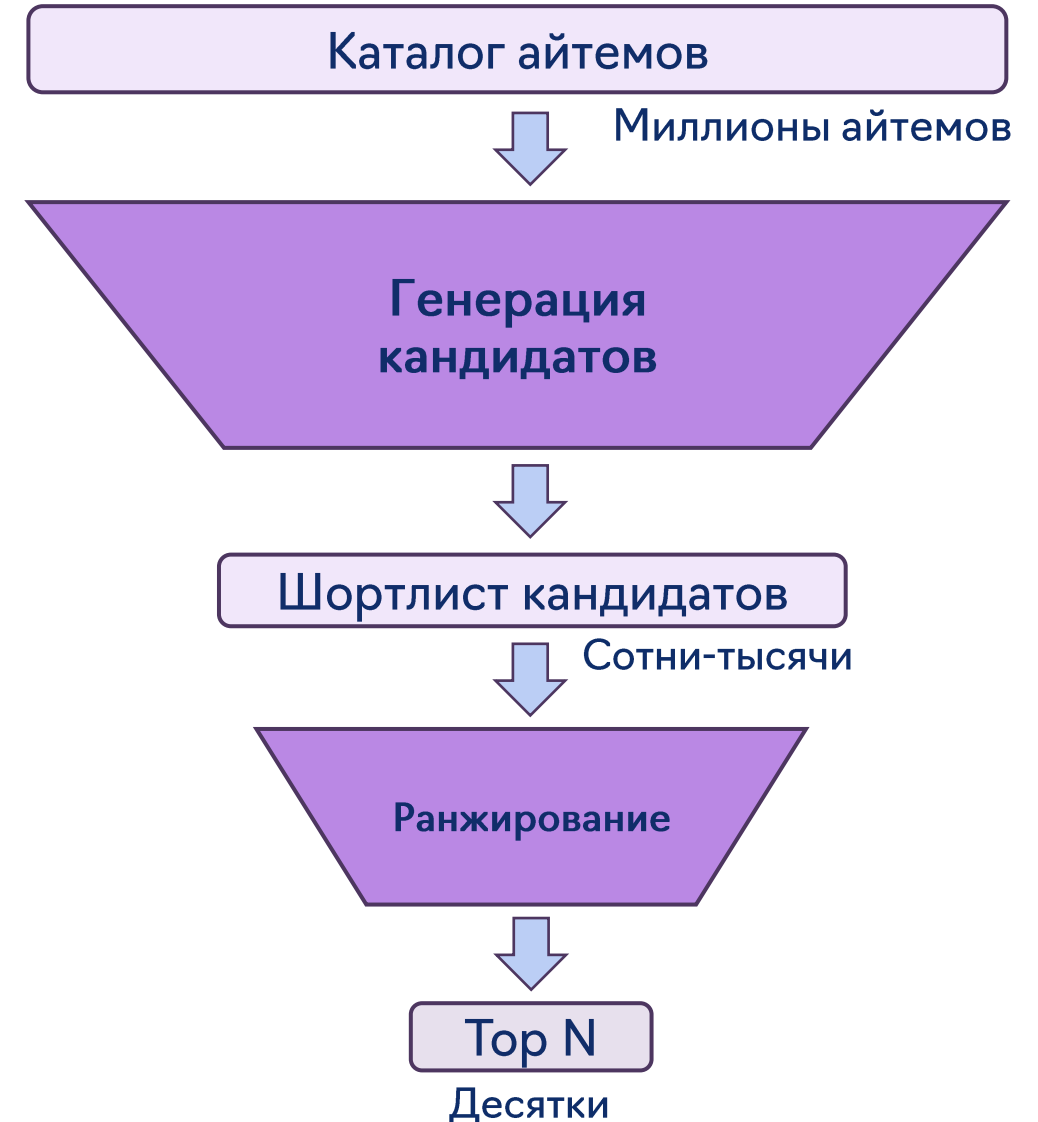
Введение в двухбашенные модели

Часть 1

Многостадийные рексистемы

Рекомендации – многостадийный процесс:

- На ранних стадиях – более быстрые и эффективные алгоритмы
- Далее – более тяжелые модели



Классические генераторы кандидатов

- **Топ популярного:** нет персонализации, не рекомендует новые айтемы
- **История пользователя:** нет discovery
- **Item-to-item:** мало персонализации и discovery, не учитывает контекст
- **User-to-item:** плохо работает на новых пользователях; не рекомендует новые айтемы
- **Матричная факторизация:** плохо для новых айтемов и пользователей, не учитывает контекст
- У всех методов кроме item-to-item – плохое качество на тяжелом хвосте

Нужны нейросети!

Двухбашенные модели

- Пользователи и айтемы **раздельно** кодируются в эмбединги:

$$g_{\theta}(u), u \in \mathcal{U} \text{ и } h_{\theta}(d), d \in \mathcal{D}$$

- Позднее связывание:** мера похожести – скалярное произведение

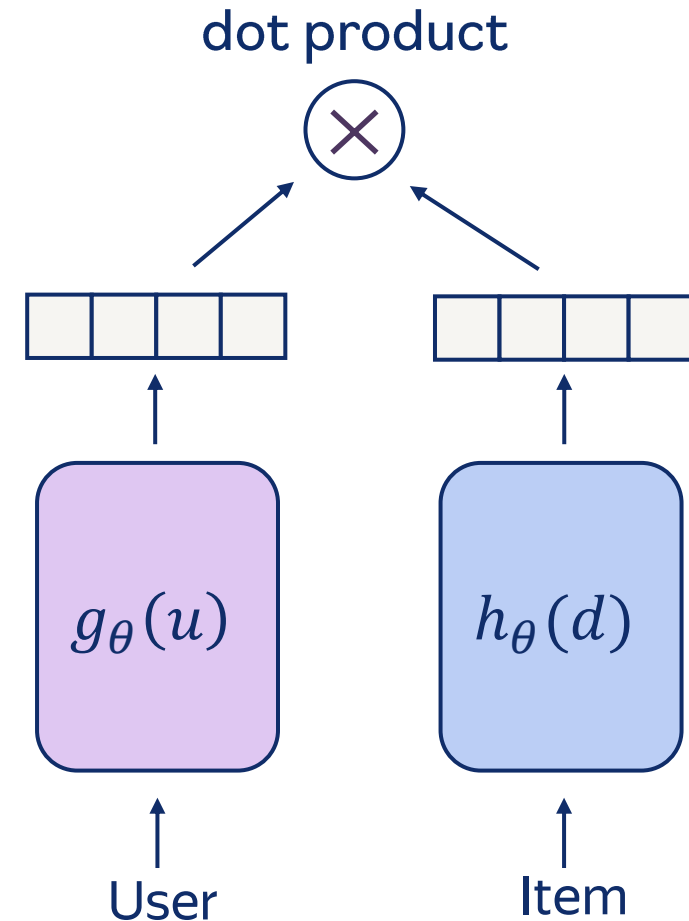
$$s_{\theta}(u, d) = \langle g_{\theta}(u), h_{\theta}(d) \rangle$$



В 2013 году появился DSSM – первая двухбашенная нейросеть для информационного поиска.

1. [Learning deep structured semantic models for web search using clickthrough data](#)

2. [Deep Neural Networks for YouTube Recommendations](#)

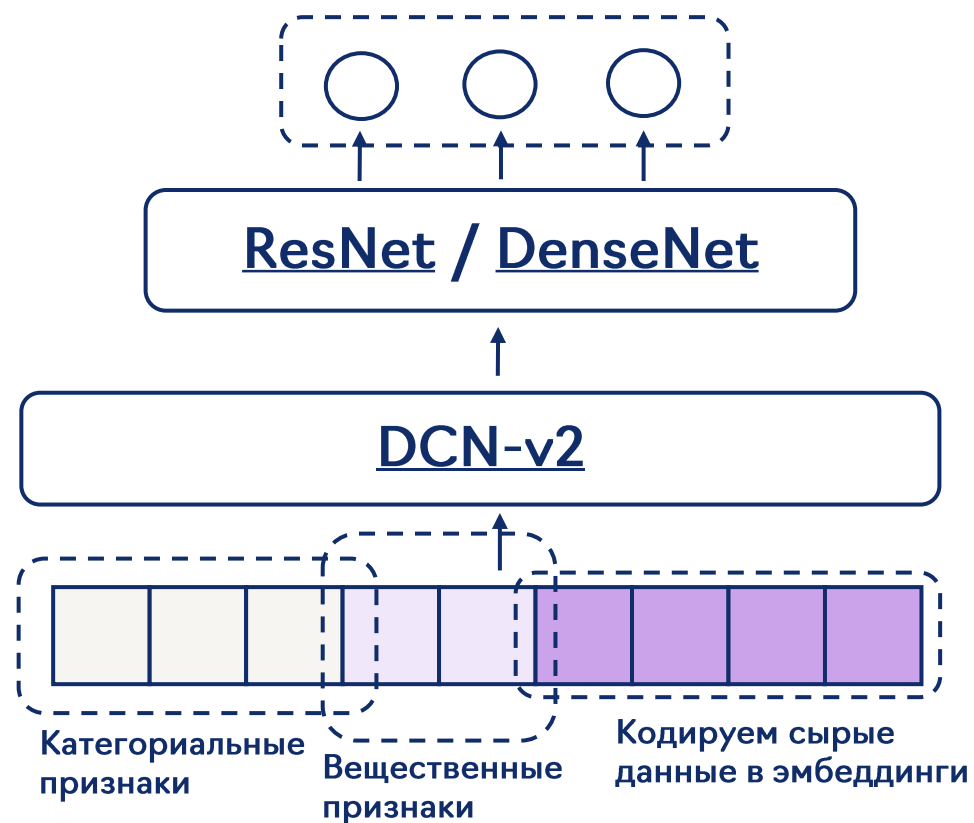


Зачем нам двухбашенность?

На стадии ранжирования используется раннее связывание:

- Подаём на вход единой нейросети всю информацию про пользователя и айтемы
- Используем специализированные нейросетевые слои для моделирования взаимодействия признаков

Не можем применить такую модель над всем каталогом.



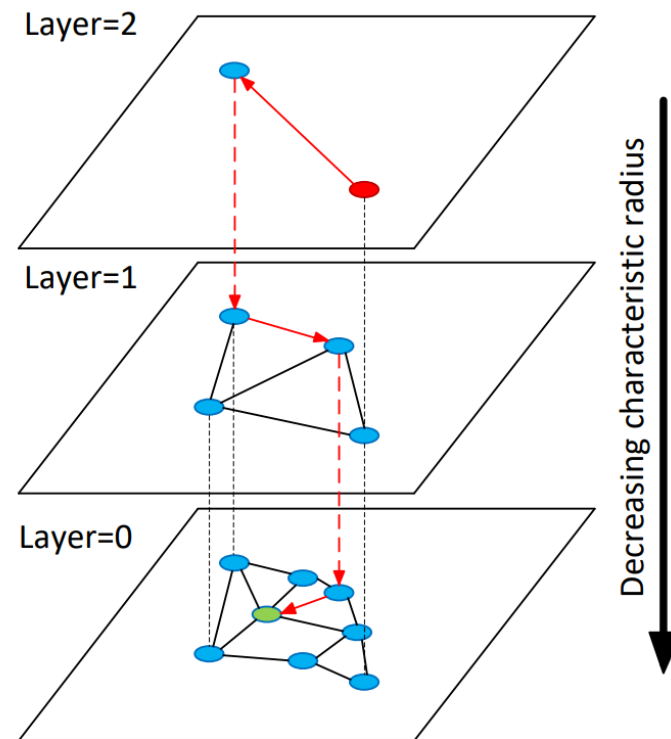
Архитектура ранжирующей нейросети.

Эффективность двухбашенных моделей

- Двухбашенность – это факторизация вычислений по пользователю и айтему:

$$s_{\theta}(u, d) = \langle g_{\theta}(u), h_{\theta}(d) \rangle$$

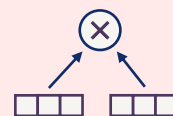
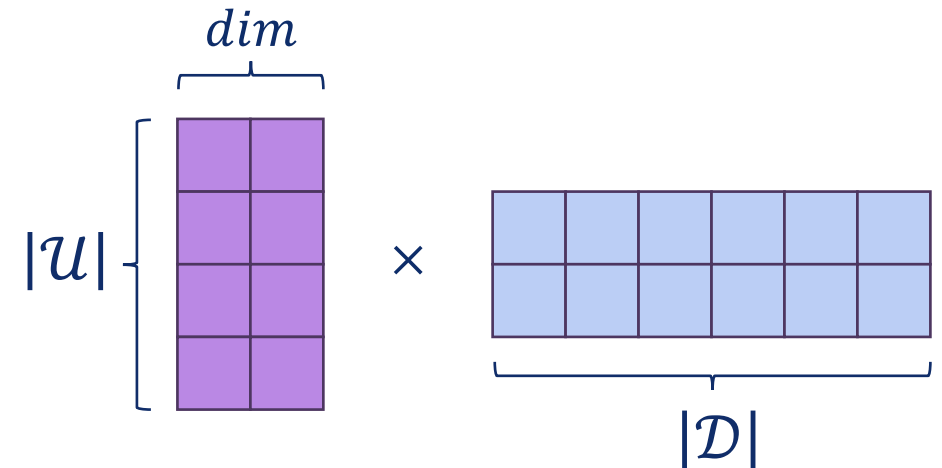
- Чтобы найти для пользователя top K кандидатов:
 1. Кодировем пользователя в эмбединг
 2. Идём в **индекс**, заранее построенный по эмбедингам айтеров
 3. Получаем приближённый top K элементов из индекса с максимальным скалярным произведением



[Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs](#)

Двухбашенная модель как низкоранговое разложение

- Пусть есть матрица релевантности $\mathcal{R} \in \mathbb{R}^{|\mathcal{U}| \times |\mathcal{D}|}$, определяющая интерес пользователей к айтемам
- Двухбашенная модель учит низкоранговое разложение $\mathcal{R} \approx V_u \times V_d^T$, в котором
 - V_u – эмбединги пользователей
 - V_d – эмбединги айтемов
- Размерность эмбединга – это информационный буттлнек
 - ограничивает ранг разложения



Матричная факторизация – это частный случай двухбашенной модели, в которой башни – это просто таблицы обучаемых векторов.

Двухбашенная модель как низкоранговое разложение

Достаточно широкий эмбединг может выучить любую функцию похожести (матрицу релевантности):

- Вектор пользователя – one hot представление

$$v_u = (0, \dots, 1, \dots, 0) \in \mathbb{R}^{|\mathcal{U}|}$$

- Вектор айтема – столбец из R , соответствующая айтему

$$v_d = (\mathcal{R}_{ud})_{u=1}^{|\mathcal{U}|}$$

- Тогда $s(u, d) = \langle v_u, v_d \rangle = \mathcal{R}_{ud}$

На практике не можем позволить себе очень большие размерности из-за ограничений по памяти и скорости

- Используем сотни-тысячи

Обучение

Часть 2

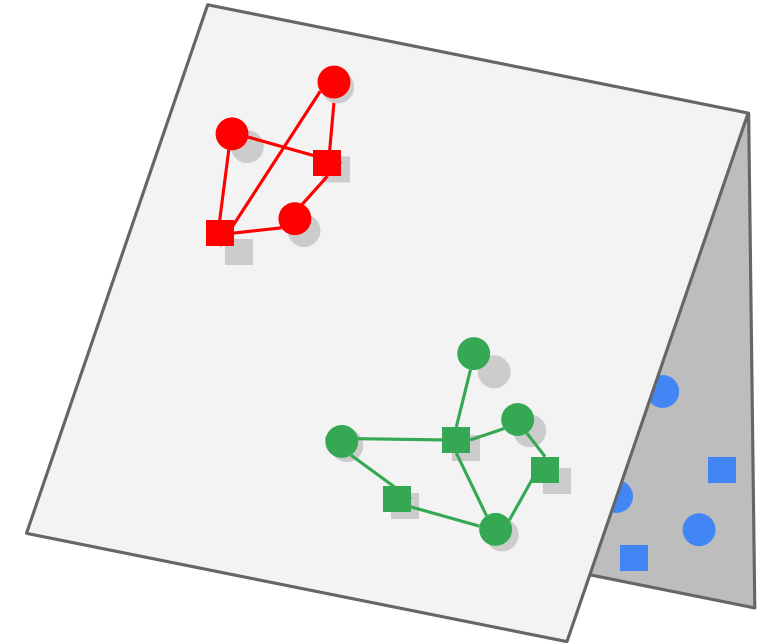
Как учатся ранжирующие модели

- Impression-aware: учимся на показанных рекомендациях
- Pointwise / pairwise обучение
- Соответствует ли область применения области обучения?

Folding

- Если рексистема работает хорошо, то в выдачах не встречается случайных айтемов
- Возникают непересекающиеся user-item группы
 - Испанцам – видеоролики на испанском, итальянцам – на итальянском
- Ранжирующий лосс учит модель делать сравнения только в рамках этих групп
- Возникает **folding** – разные user-item группы накладываются (англ. “to fold”) друг на друга в векторном пространстве

Нам нужна способность глобального сравнения: для пользователя сравнивать все айтемы, а не только внутри “фолдов”



Из [Google for Developers, Recommender Systems](#)

[Folding: Why Good Models Sometimes Make Spurious Recommendations](#)

Одностадийная рексистема

- Как должна обучаться одностадийная рексистема?
- Будет ли folding при обучении на ранжирование?

Softmax model

Перейдём от $P(y = 1 \mid user, item) \rightarrow P(item \mid user, y = 1)$

- Раньше: пользователю u показали видео i . Поставил ли он лайк ($y = 1$)?
- Теперь: знаем, что пользователь u поставил лайк ($y = 1$) какому-то видео i . Что это за видео i могло быть?

Это задача экстремальной классификации, в которой каждому айтому из каталога соответствует свой класс.

Softmax model

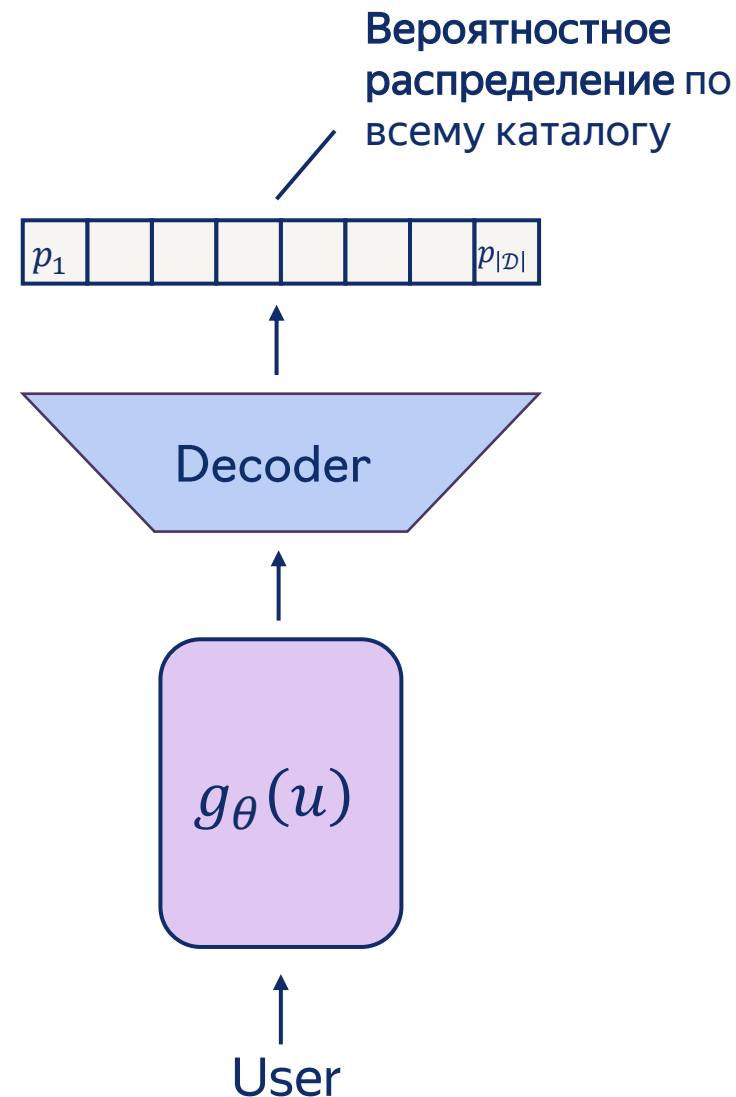
- На входе в модель – пользователь
- На выходе – вероятностное распределение по всем айтемам

$$P(d \mid u) := P(d \mid u, y = 1)$$

Для моделирования вероятностей используем softmax:

$$P(d \mid u) = \frac{e^{s_{\theta}(u, d)}}{\sum_{d' \in \mathcal{D}} e^{s_{\theta}(u, d')}}$$

где $s_{\theta}(u, d)$ – функция похожести

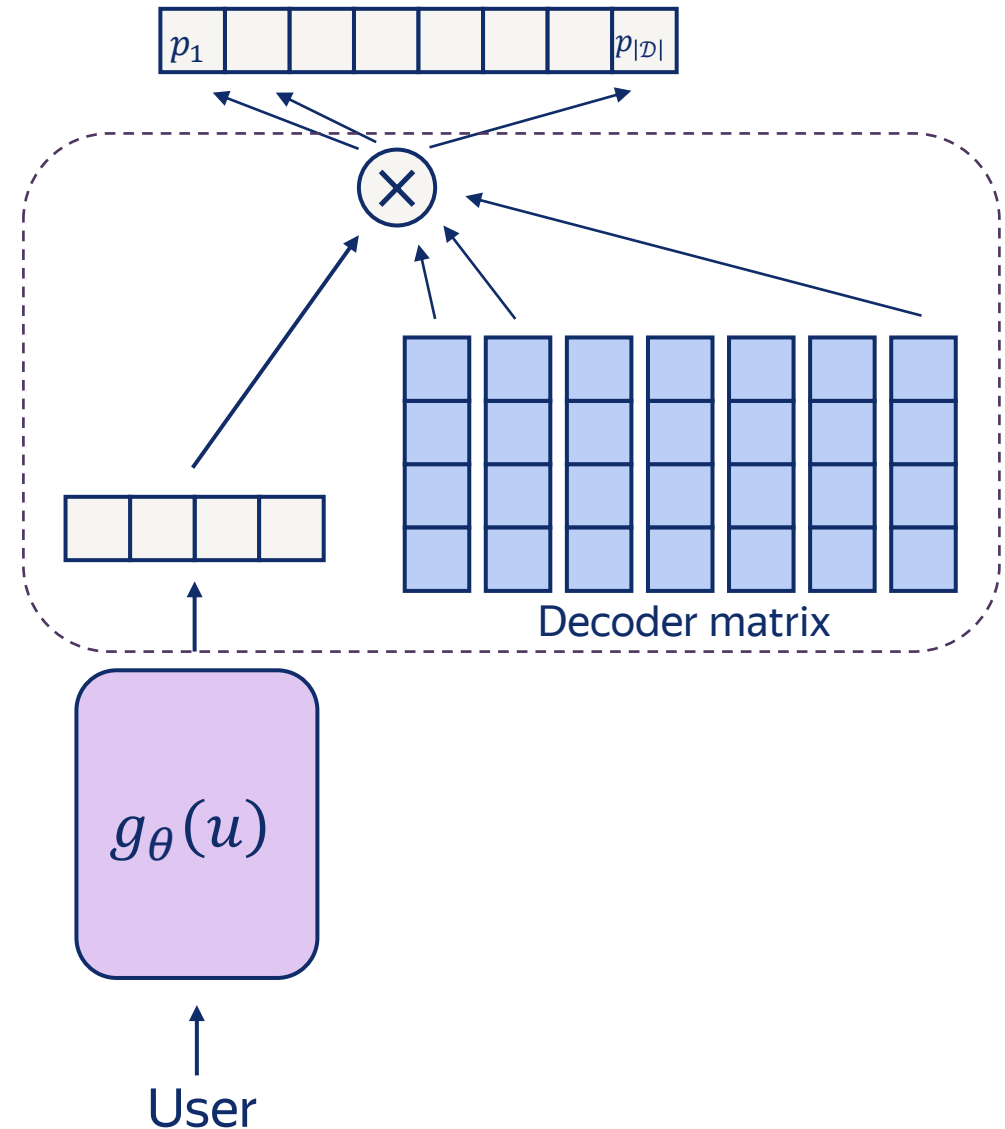


Softmax model as a two-tower model

- Получаем распределение с помощью умножения на декодер матрицу V :

$\bar{p} = \text{softmax}(g_{\theta}(u)V^T)$, то есть $s_{\theta}(u, d) = \langle g_{\theta}(u), v_d \rangle$, где $g_{\theta}(u)$ – вектор-строка

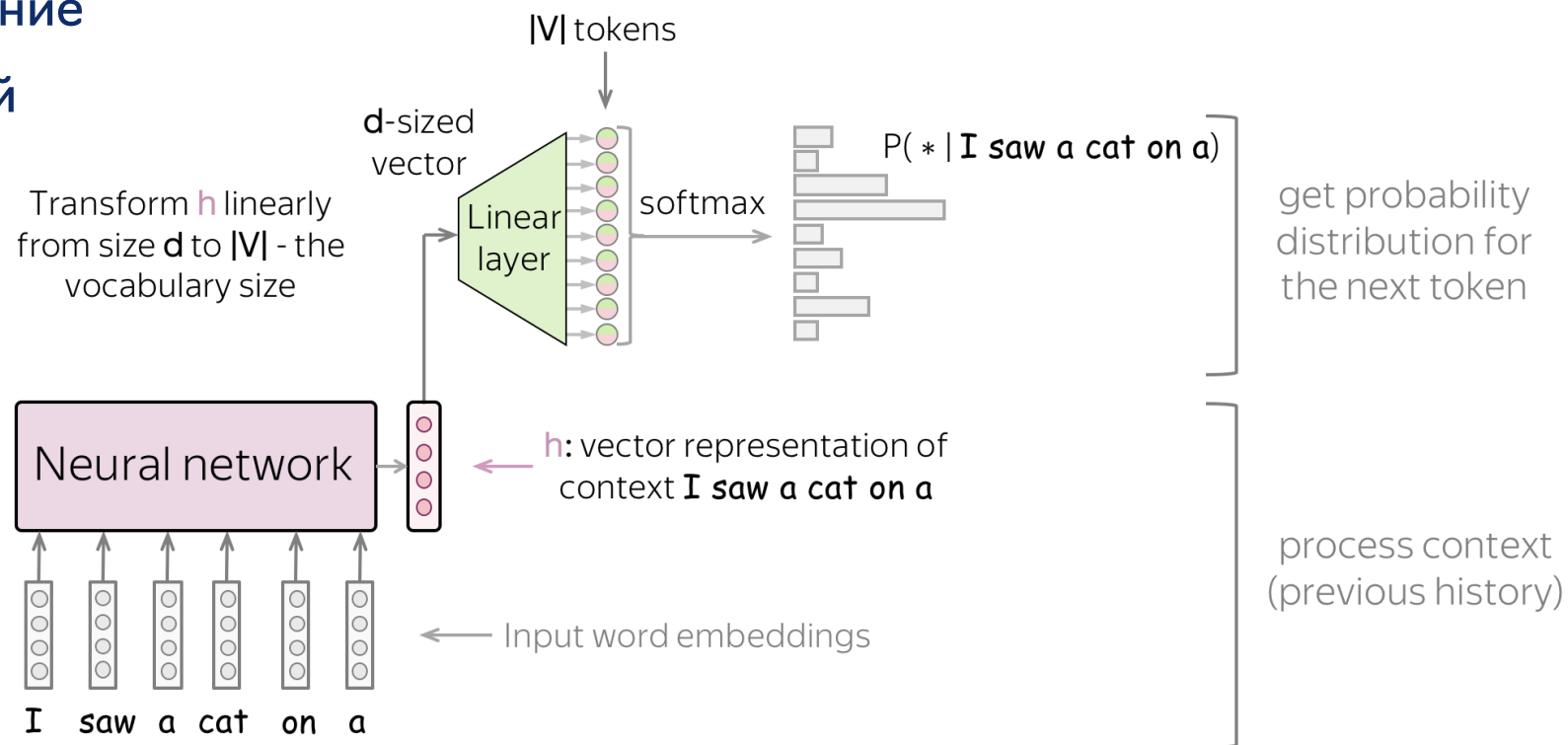
- Декодер – матрица обучаемых эмбедингов айтемов
- Это двухбашенная модель:
 - Левая часть – энкодер пользователя $g_{\theta}(u), u \in \mathcal{U}$
 - Правая часть – обучаемые эмбединги айтемов $v_1, \dots, v_{|\mathcal{D}|}$



LLM as a two-tower model

LLM – это softmax model:

- Левая часть: уже обработанное предложение
- Правая часть: следующий токен



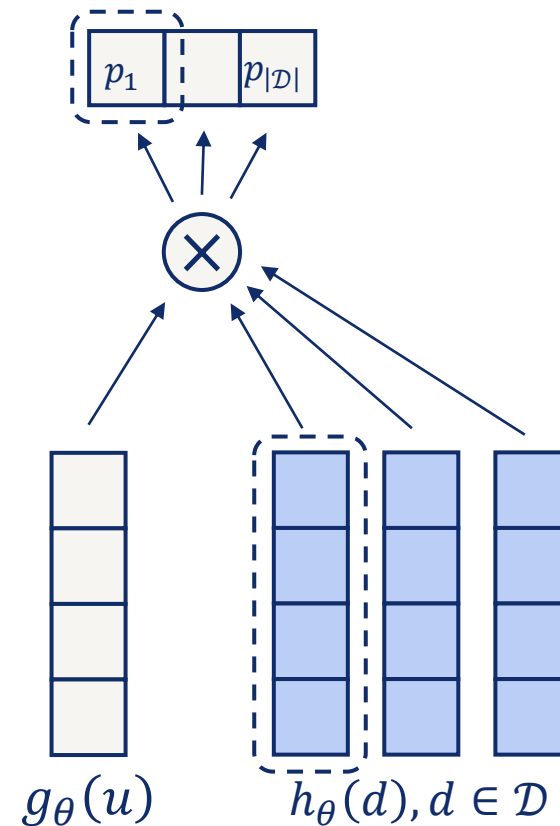
Softmax loss

Максимизируем правдоподобие реальных положительных взаимодействий:

$$\mathcal{L}_{\text{softmax}}(u, d_+) = -\log P_{\theta}(d_+ \mid u) = -\log \frac{e^{s_{\theta}(u, d_+)}}{\sum_{d \in \mathcal{D}} e^{s_{\theta}(u, d)}}$$

Полный софтмакс не вычислим на больших масштабах:

- Каталоги больше 10^5 – слишком дорогой знаменатель
- И по вычислениям, и по памяти невозможно суммировать по всему каталогу



Sampled softmax loss

Sampled softmax loss аппроксимирует знаменатель с помощью сэмплирования негативов:

$$\mathcal{L}_{\text{sampled}}(u, d_+, N) = -\log \frac{e^{s_\theta(u, d_+)}}{e^{s_\theta(u, d_+)} + \sum_{d \in N} e^{s_\theta(u, d)}}$$

- Где $N = \{d_1, \dots, d_{|N|} : d_i \sim Q(d)\}$ – сэмплированные негативы
- Это тоже задача классификации, но с подмножеством каталога в качестве классов

Contrastive learning & InfoNCE

Contrastive learning – сближаем похожие объекты и отдаляем непохожие в рамках обучения модели.

Пусть есть батч из положительных пар вида $(u_1, d_1), \dots, (u_n, d_n)$.
Тогда:

$$\mathcal{L}_{\text{InfoNCE}}(u_i, d_i) = -\log \frac{e^{\cos(u_i, d_i)/\tau}}{\sum_{j=1}^n e^{\cos(u_j, d_j)/\tau}}$$

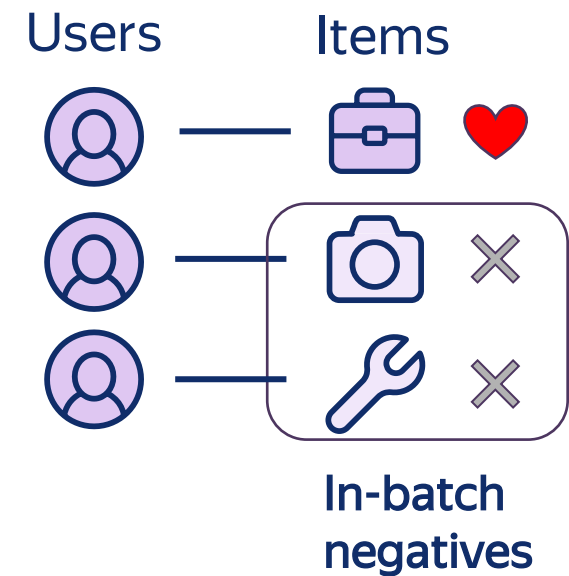
InfoNCE:

- $s_\theta(u, d) = \cos(u, d)/\tau$, где $\tau > 0$ – это температура
- In-batch negatives

In-batch negatives

В батче n положительных пар ($user, item$):

- Для каждой пары переиспользуем в качестве негативов позитивы из других пар
- Это очень эффективно:
 - Не нужно перевычислять эмбединги для негативов
 - Можно переиспользовать негативы с прошлых батчей (накапливать очередь, **memory queue**)
 - При распределенном обучении можно собирать **cross-device** негативы с других видеокарт



1. [Momentum Contrast for Unsupervised Visual Representation Learning](#)
2. [Cross-Batch Negative Sampling for Training Two-Tower Recommendations](#)

Про косинус

- Обучение с косинусом – более стабильное
 - Логиты ограничены по масштабу
- Если объекты достаточно хорошо кластеризованы, появляется линейная сепарабельность
- Нормы айтемов имеют тенденцию выучивать популярность
 - Косинус убирает влияние нормы (понижаем popularity bias)

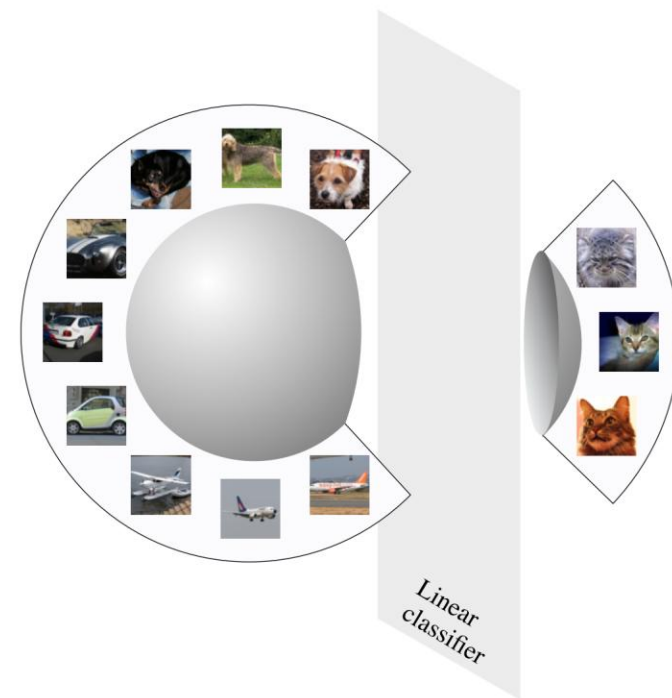


Figure 2: **Hypersphere:** When classes are well-clustered (forming spherical caps), they are linearly separable. The same does not hold for Euclidean spaces.

[Understanding Contrastive Representation Learning through Alignment and Uniformity on the Hypersphere](#)

Температура

$$s_{\theta}(u, d) = \cos(u, d) / \tau$$

- Если используем косинус, то логиты ограничены $[-1; 1]$
- Такой софтмакс будет очень “гладким”
- Температура позволяет отмасштабировать логиты до большего интервала значений
- Чем меньше температура, тем больше интервал значений

Логиты	τ	P(positive)	P(negative)
1, -1, -1, -1, -1	1	0.65	0.088
1, -1, -1, -1, -1	0.1	~ 1.0	~ 0

Температура как мера неопределенности

- Для заданных $(s_1, \dots, s_n)$ распределение с температурой можно записать следующим образом:

$$p_k = \frac{e^{s_k/\tau}}{\sum_{i=1}^K e^{s_i/\tau}} = \frac{e^{-E_k/\tau}}{\sum_{i=1}^K e^{-E_i/\tau}} = \frac{e^{-E_k/\tau}}{Z}$$

- Это Boltzmann / Gibbs distribution
- Температура – мера неопределенности
 - Чем больше температура, тем больше энтропия распределения

Температура как мера неопределенности

Температура как мера неопределенности

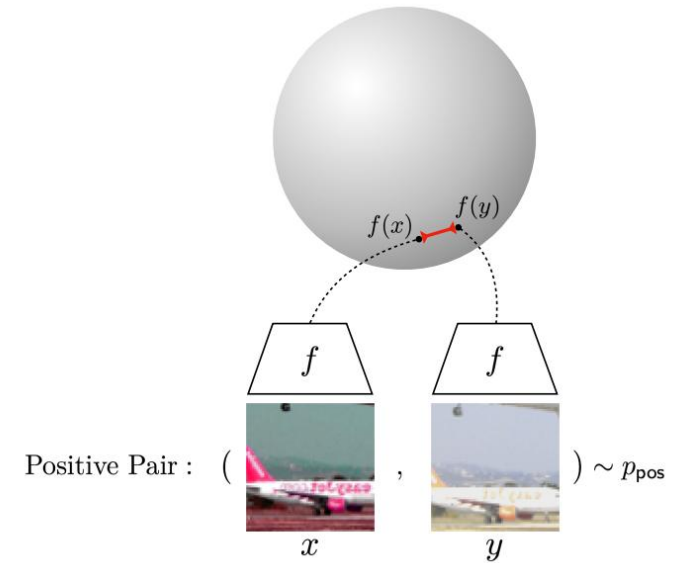
Свойства софтмакса

- Софтмакс неявно майнит хард негативы
- Негативы конкурируют между собой за вклад в градиент

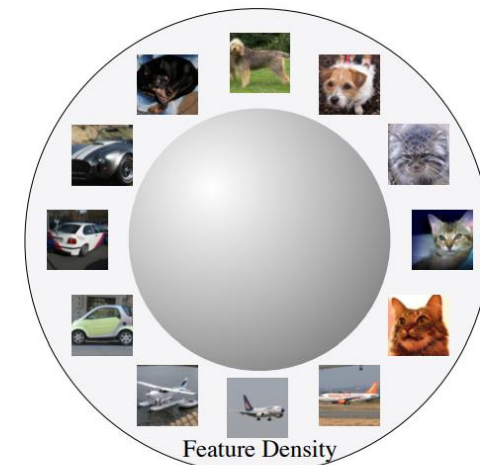
Свойства температуры

- **Alignment** – похожие объекты должны быть близко в пространстве
- **Uniformity** – объекты должны равномерно покрывать сферу

При уменьшении температуры увеличиваем uniformity (и уменьшаем alignment).



Alignment: Similar samples have similar features.
(Figure inspired by Tian et al. (2019).)



Uniformity: Preserve maximal information.

[Understanding Contrastive Representation Learning through Alignment and Uniformity on the Hypersphere](#)

Обучаемая температура

- Можно зафиксировать температуру
- Можно сделать расписание для температуры (постепенно её уменьшать) – аналог curriculum learning
- Можно сделать обучаемую температуру – тогда это **неуверенность модели**

Negative sampling

Softmax loss “майнит” хард негативы по всему каталогу – вклад в градиент больше у тех айтеров, в которых модель ошибается

- Если не можем взять все айтеры, надо взять те, у которых будет большой вклад в градиент

Основные источники негативов:

- Uniform
- In-batch negatives
- Mixed negative sampling

Uniform negatives

Uniform negatives, $d \sim \text{Uniform}(\mathcal{D})$:

- Равномерное сэмплирование из каталога
- Очень “простые” для модели
- Дают небольшой вклад в градиент
- Нужно большое количество, чтобы выучить модель

In-batch negatives

In-batch негативы эквивалентны сэмплированию из униграммного (частотного) распределения

- Частота попадания айтема в батч пропорциональна его общей популярности
- Это более сложные негативы по сравнению с равномерными, $f_{\theta}(u, d)$ в среднем больше для случайных пользователей
- Нужно меньше негативов, чтобы обучить модель

LogQ Correction

Часть 3

Bias of in-batch negatives

In-batch негативы приводят к систематическому смещению:

- Популярные айтемы часто появляются в негативах
- один раз как позитив и batch size - 1 раз как негатив
- Модель видит корреляцию между популярностью и “негативностью” → начинает штрафовать популярные объекты

Смещение оценки градиента

Importance sampling

Вывод $\log Q$ коррекции

LogQ коррекция

Sampled softmax loss с logQ коррекцией:

$$\mathcal{L}_{\text{logQ}}(u, d_+, N) = -\log \frac{e^{s_\theta(u, d_+) - \log Q(d_+)}}{e^{s_\theta(u, d_+) - \log Q(d_+)} + \sum_{d \in N} e^{s_\theta(u, d) - \log Q(d)}}$$

- Q – распределение, из которого сэмплируем негативы
- N – negative pool
- Интуиция: logQ поправка штрафует айтемы за популярность при обучении

[1. Quick Training of Probabilistic Neural Nets by Importance Sampling](#)

[2. Sampling-Bias-Corrected Neural Modeling for Large Corpus Item Recommendations](#)

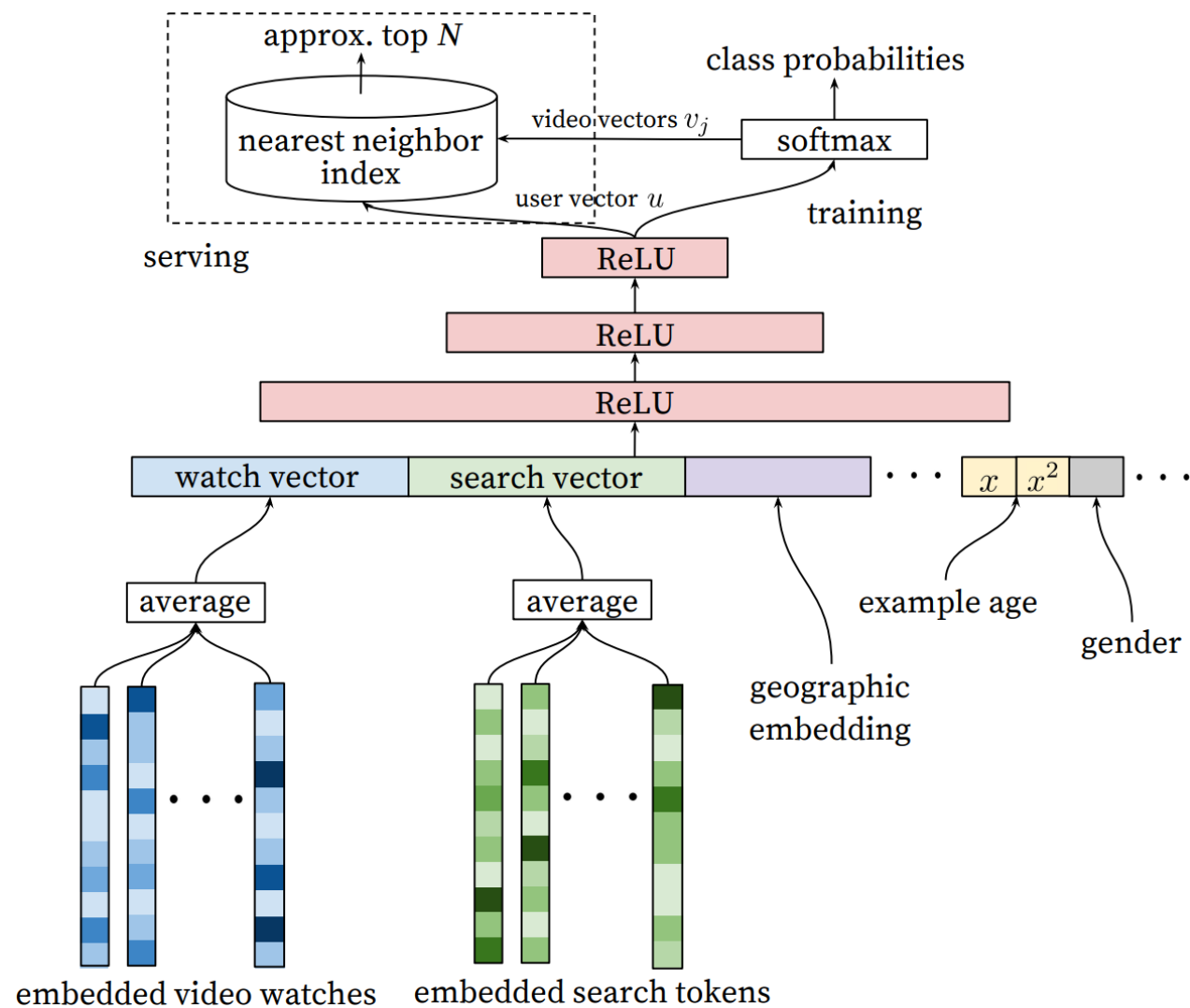
Ошибка в выводе logQ коррекции

Correcting the LogQ Correction

$$\mathcal{L}_{\log Q}^{\text{corrected}}(u, d_+, N) = -w_{ud_+} \log \frac{e^{s_\theta(u, d_+)}}{\sum_{d \in N} e^{s_\theta(u, d) - \log Q(d)}}$$

- Где $w_{ud_+} = \text{stopgrad}(1 - P(d \mid u))$ и $P(d \mid u) \approx \frac{e^{s_\theta(u, d_+)}}{e^{s_\theta(u, d_+)} + \frac{1}{|N|} \sum_{d \in N} e^{s_\theta(u, d) - \log Q(d)}}$
- В знаменателе больше не участвует позитив
- Вес сэмпла = вероятность, что модель на нём ошибётся

YoutubeDNN



Summary

Обсудили:

- Как выглядят двухбашенные модели, зачем они нужны и как их обучать
- Свойства софтмакса, косинуса, температуры
- Источники негативов
- Проблемы униграммного сэмплирования негативов и $\log Q$ коррекцию